Reviewer Pack — Matroid Rank Deficit in Monotone DNF Depth for k-CLIQUE

Entry 9d5d3a5ea1bf · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 17:23:25 UTC

Matroid Rank Deficit in Monotone DNF Depth for k-CLIQUE

Entry ID: `9d5d3a5ea1bf` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-08 17:23:25 UTC

1. Conjecture

Field A (mathematical branch): Matroid Theory **Field B** (complexity object): Monotone Circuit Depth

Statement:

> For any monotone DNF formula F with n variables, let M_F be the matroid whose circuits are the minimal non-empty terms of F. Define rank_deficit(F) = rank(M_F) - log_2(size(F)). Then rank_deficit(F) ≤ 2 log n for all F with size(F) ≤ n^3, and rank_deficit(k-CLIQUE) ≥ Ω(n^{1/2}) for k ≥ 3.

Rationale (proposer's reasoning):

> Matroid rank captures structural dependencies in DNF terms, which may expose limitations in representing k-CLIQUE. The deficit metric isolates the 'wastefulness' of DNF representations, linking matroid invariants to circuit complexity through combinatorial optimization.

Taxonomy category: `MONOTONE_CLIQUE` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `005326a000c5ea27`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE

Barrier	Verdict	Confidence	LLM_1	LLM_2
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=0, elapsed=2.8s

5.1 Generated Python source

```python
import random
import math
from itertools import combinations

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_dnf(n, max_terms):
        terms = []
        for _ in range(random.randint(1, max_terms)):
            term = [random.choice([0, 1]) for _ in range(n)]
            if sum(term) > 0:
                terms.append(term)
        return terms

    def size(dnf):
        return len(dnf)

    def matroid_rank(circuits):
        rank = 0
        independent_sets = []
        for circuit in circuits:
            is_independent = True
            for s in independent_sets:
                if all(x == y or x == 0 or y == 0 for x, y in zip(circuit, s)):
                    is_independent = False
                    break
            if is_independent:
                independent_sets.append(circuit)
                rank += 1
        return rank

    def rank_deficit(dnf):
        circuits = dnf
        M = matroid_rank(circuits)
        return M - math.log2(size(dnf))

    n = random.randint(5, 40)
    max_terms = n**3
    dnf = generate_dnf(n, max_terms)
```

```
    metric_value = rank_deficit(dnf)
    instances_tested = 1
    conjecture_holds = True
    counterexample = ""

    if metric_value > 2 * math.log(n):
        conjecture_holds = False
        counterexample = "DNF violates upper bound"

    return {
        "metric_name": "rank_deficit",
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if len(sys.argv) > 1 else [2**i - 1 for i in range(5, 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value)**2 for r in results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fractior
    elif sum(1 for r in results if not r["conjecture_holds"]) / len(results) < 0.2:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fractior
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
: ''}
TRIAL: {'metric_name': 'rank_deficit', 'metric_value': -13.200208961930477, 'instances_tested': 1, '
TRIAL: {'metric_name': 'rank_deficit', 'metric_value': -11.982637133669424, 'instances_tested': 1, '
TRIAL: {'metric_name': 'rank_deficit', 'metric_value': -10.915505962231931, 'instances_tested': 1, '
TRIAL: {'metric_name': 'rank_deficit', 'metric_value': -8.280770770130603, 'instances_tested': 1, 'c
TRIAL: {'metric_name': 'rank_deficit', 'metric_value': -4.285402218862249, 'instances_tested': 1, 'c
TRIAL: {'metric_name': 'rank_deficit', 'metric_value': -11.553869243196068, 'instances_tested': 1, '
TRIAL: {'metric_name': 'rank_deficit', 'metric_value': -11.440350119200563, 'instances_tested': 1, '
TRIAL: {'metric_name': 'rank_deficit', 'metric_value': -8.361943773735241, 'instances_tested': 1, 'c
TRIAL: {'metric_name': 'rank_deficit', 'metric_value': -12.444108996147115, 'instances_tested': 1, '
TRIAL: {'metric_name': 'rank_deficit', 'metric_value': -10.98903963696239, 'instances_tested': 1, 'c
```

```
TRIAL: {'metric_name': 'rank_deficit', 'metric_value': -13.283667167491501, 'instances_tested': 1, '
TRIAL: {'metric_name': 'rank_deficit', 'metric_value': -11.659995892429977, 'instances_tested': 1, '
TRIAL: {'metric_name': 'rank_deficit', 'metric_value': -11.086136225027309, 'instances_tested': 1, '
RESULT: SUPPORTED mean=-10.735960113292682 std=2.3443356317573154 support_fraction=1.0
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> parse_fail | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	40766
2	preregistration	ollama_remote	qwen3:8b	0	0	24378
3	novelty	ollama_remote	qwen3:8b	0	0	21022
4	novelty	ollama_remote	qwen3:8b	0	0	14279
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18049
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8234
7	critic	ollama_remote	qwen3:8b	0	0	30315
8	judge	ollama_remote	qwen3:8b	0	0	21293

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 178335 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in `research/audit/9d5d3a5ea1bf.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/9d5d3a5ea1bf.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/9d5d3a5ea1bf.tar.gz` (if generated)